

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Системи штучного інтелекту»

Робота №1

Тема «Розпізнавання образів на основі метричної класифікації»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Калюжна О.С.

2025

МЕТА

Вивчити та засвоїти на практиці метричні методи розпізнавання образів у просторі ознак, навчитися створювати програмні засоби, що реалізують методи метричної класифікації, порівняти результати роботи метричної класифікації з іншими видами класифікації.

ЗАВДАННЯ

Частина 1.

1. Ознайомитися з конспектом лекцій та рекомендованою літературою. На мові програмування Python написати програму, що реалізує процедури для навчання та емуляції за методом еталонів.

2. Згідно з номером студента за журналом для відповідного номера варіанта V сформувати навчаючу вибірку $\langle x, y \rangle$ обсягом S екземплярів x^s , $s=1, 2, \dots, S$, що характеризуються N ознаками x_j^s , $j=1, 2, \dots, N$, та зіставити кожному екземпляру значення цільової ознаки y_s :

$$x_j^s = \begin{cases} jV - 0,1s, & j = 1, 5, 9, \dots, \\ 0,01jV^{-1} + 0,3s, & j = 2, 4, 6, \dots, \\ j\text{rand}, & j = 3, 7, 11, \dots; \end{cases} \quad y^s = \begin{cases} 0, & (x_1^s)^2 + (x_2^s)^2 < V^2 + 0,04S^2, \\ 1, & (x_1^s)^2 + (x_2^s)^2 \geq V^2 + 0,04S^2; \end{cases}$$
$$S = \begin{cases} 10V, & V < 10, \\ 5V, & 10 \leq V < 20, \\ 3V, & V \geq 20; \end{cases} \quad N = \begin{cases} 5V, & V < 7, \\ 4V, & 7 \leq V < 10, \\ 3V, & 10 \leq V < 20, \\ 2V, & V \geq 20; \end{cases}$$

де rand - випадкове число в діапазоні $[0, 1]$.

3. Виконати нормування навчаючої вибірки даних.

4. На основі пронормованої вибірки побудувати розпізнаючу модель за методом еталонів, тобто визначити координати центрів класів (еталонів) у просторі ознак C_j^q , де q - номер класу, j - номер ознаки. Зафіксувати час навчання.

5. На основі побудованої моделі для екземплярів навчаючої вибірки виконати розпізнавання, тобто визначити розрахункові номери класів ys^* . Зафіксувати час розпізнавання.

6. Обчислити помилку розпізнавання, визначити ймовірність прийняття правильного рішення та ймовірність прийняття помилкового рішення для побудованої моделі.

7. У попередньо сформованій вибірці залишити тільки одну ознаку, номер якої у попередній вибірці дорівнює V , а також у центрів еталонів класів залишити тільки V -ту координату C_qV .

8. Для нової вибірки та нових еталонів виконати розпізнавання, тобто визначити розрахункові номери класів ys^* . Зафіксувати час розпізнавання.

9. Обчислити помилку розпізнавання, визначити ймовірність прийняття правильного рішення та ймовірність прийняття помилкового рішення для нової моделі.

10. Порівняти результати проведених експериментів, зробити висновки щодо впливу параметрів навчаючої вибірки на характеристики процесів навчання та розпізнавання.

Частина 2.

1. Описати вибірку, надати короткий опис наявним в вибірці ознакам, проаналізувати їх, визначити малозначущі і видалити їх (за наявності).

2. Дослідити наявність пропущених даних, їх кількість. За потреби усунути нестачу будь-яким прийнятним методом. Обґрунтувати вибір.

3. Виконати розпізнавання наступними алгоритмами з пакету «Sklearn» і порівняти їх з результатами метричної класифікації (методом найближчих сусідів):

- методом найближчих сусідів (KNeighborsClassifier);
- випадковий ліс (RandomForestClassifier);
- метод логістичної регресії (LogisticRegression);
- метод опорних векторів (SVC).

4. Результати розпізнавання відобразити у вигляді графіку, проаналізувати їх, зробити припущення як їх можна покращити.

ТЕОРІЯ

Пакет `scikit-learn` призначений для формування моделей розпізнавання образів та їх використання.

Вибірку даних потрібно розбити на навчальну та тестову функцією `train_test_split`.

Непотрібні ознаки можна прибрати функцією `drop()`.

Метод еталонів реалізується моделлю `KMeans`. Вона приймає кількість кластерів та шукає їх центри. Тоді відносить кожен екземпляр до певного кластеру.

Навчання моделі проводиться функцією `fit()`. Розпізнавання виконується функцією `predict()`.

Нормування вибірки можна виконати функцією `normalize()`.

Засікти час виконання можна функцією `time.time()` або `time.perf_counter()`.

Отримати помилку розпізнавання можна функцією `score()`.

ВИКОНАННЯ

Частина 1

```
import os
import random
import time
import pandas as pd
import numpy as np

from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, normalize

current_folder = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(current_folder, 'train.csv')
df = pd.read_csv(file_path)

# 1.1 Еталони

needed='Name, Age, Survived'.split(',')
df=df[needed].dropna()
le=LabelEncoder()

target_feature=df.Survived.to_numpy()
df=df.drop(['Survived'],axis=1)
df=df.drop(['Name'],axis=1)

X_train,X_test,y_train,y_test=train_test_split(df,target_feature,test_size=0.3)
model=KMeans(n_clusters=3)
model.fit(X_train)
print(f'{model.cluster_centers_ = }')
prediction_results=model.predict(X_test)
print(f'{prediction_results = }')

# 1.2 Вибірка

def get_target_feature(row: list[int]):
    if row[0]**2+row[1]**2 < variant**2+0.04*size**2: return 0
    elif row[0]**2+row[1]**2 >= variant**2+0.04*size**2: return 1

variant=19
size=5*variant
number_of_features=3*variant

ones_range=range(1,size,4)
twos_range=range(2,size,4)
threes_range=range(3,size,4)
features_input=[]
targets_input=[]
for data_row in range(1,size):
    row=[]
    for feature_col in range(1,number_of_features):
        if feature_col in ones_range:
            result=feature_col*variant-0.1*data_row
```

```

        elif feature_col in twos_range:
            result=0.01*feature_col*variant** -1
        elif feature_col in threes_range:
            result=feature_col*random.random()
        row.append(result)

    targets_input.append(get_target_feature(row=row))
    features_input.append(row)
features=np.array(features_input)
targets=np.array(targets_input)
print(f'{features = }')
print(f'{targets = }')

# 1.3 Нормування

normalized_features=normalize(features)
print(f'{normalized_features = }')

# 1.4 Побудова моделі

features_train, features_test, targets_train, targets_test=train_test_split(normalized_features, targets)

start_time=time.perf_counter()
model=KMeans(n_clusters=number_of_features)
model.fit(features_train)
end_time=time.perf_counter()
learning_time=end_time-start_time
print(f'{learning_time = }')
print(f'{model.cluster_centers_ = }')

# 1.5 Розпізнавання

start_time=time.perf_counter()
result=model.predict(features_test)
print(f'{result = }')
end_time=time.perf_counter()
prediction_time=end_time-start_time
print(f'{prediction_time = }')

# 1.6 Помилка

first_prediction_error=model.score(features_test)
print(f'{first_prediction_error = }')
first_error_probability=first_prediction_error/size*number_of_features
first_right_probability=1-first_error_probability
print(f'{first_error_probability = }')
print(f'{first_right_probability = }')

# 1.7 Нова вибірка

new_features=[row[variant] for row in features_input]
print(f'{new_features = }')
new_center=int(result[variant])
print(f'{new_center = }')
new_target=get_target_feature(row=new_features)
print(f'{new_target = }')

```

```
# 1.8 Розпізнавання
```

```
new_features=np.array(new_features).reshape(-1,1)
model=model.fit(new_features)
result=model.predict(new_features)
print(f'{result = }')
```

```
# 1.9 Помилка
```

```
prediction_error=model.score(new_features)
print(f'{prediction_error = }')
error_probability=prediction_error/len(new_features)*1
right_probability=1-error_probability
print(f'{error_probability = }')
print(f'{right_probability = }')
```

```
# 1.10 Порівняння
```

```
prediction_error_difference=abs(first_prediction_error-
prediction_error)
error_probability_difference=abs(first_error_probability-
error_probability)
right_probability_difference=abs(first_right_probability-
right_probability)
print(f'{prediction_error_difference = }')
print(f'{error_probability_difference = }')
print(f'{right_probability_difference = }')
```

```

model.cluster_centers_ = array([[11.64536   ],
 [49.33613445],
 [27.94705882]])
prediction_results = array([2, 0, 1, 0, 0, 2, 2, 2, 1, 2, 1, 2, 0, 2, 0, 2, 1, 2, 1, 2, 2, 1,
 0, 0, 2, 1, 0, 1, 2, 0, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 1, 2, 2, 2,
 1, 1, 2, 2, 2, 1, 2, 1, 1, 2, 2, 1, 0, 0, 2, 0, 0, 2, 2, 2, 1, 2,
 0, 2, 1, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 1, 2, 0, 1, 1, 2, 2,
 1, 2, 2, 2, 1, 2, 1, 0, 0, 1, 2, 1, 2, 2, 1, 1, 2, 1, 0, 2, 0,
 0, 1, 1, 2, 1, 0, 0, 0, 0, 2, 2, 1, 2, 1, 0, 2, 2, 1, 2, 2, 2,
 1, 2, 1, 2, 2, 0, 2, 2, 2, 0, 2, 2, 1, 2, 0, 2, 2, 1, 1, 1, 2, 2,
 1, 2, 2, 2, 2, 1, 2, 1, 2, 2, 0, 1, 2, 2, 2, 1, 2, 2, 0, 2, 1, 1,
 0, 0, 2, 0, 1, 2, 1, 1, 0, 2, 0, 2, 2, 2, 0, 2, 2, 2, 2, 1, 0, 1,
 2, 2, 2, 2, 1, 2, 1, 2, 1, 2, 2, 2, 0, 2, 2, 2, 2], dtype=int32)
features = array([[1.89000000e+01, 1.05263158e-03, 5.70505015e-01, ...,
 2.84210526e-02, 1.16954017e+00, 1.16954017e+00],
 [1.88000000e+01, 1.05263158e-03, 2.73384693e+00, ...,
 2.84210526e-02, 4.68819184e+01, 4.68819184e+01],
 [1.87000000e+01, 1.05263158e-03, 2.73193176e+00, ...,
 2.84210526e-02, 4.39340314e+01, 4.39340314e+01],
 ...,
 [9.80000000e+00, 1.05263158e-03, 2.18697379e-01, ...,
 2.84210526e-02, 1.10972475e+00, 1.10972475e+00],
 [9.70000000e+00, 1.05263158e-03, 1.72854307e+00, ...,
 2.84210526e-02, 3.85877790e+01, 3.85877790e+01],
 [9.60000000e+00, 1.05263158e-03, 1.57291972e+00, ...,
 2.84210526e-02, 1.40456038e+01, 1.40456038e+01]], shape=(94, 56))
targets = array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0])
normalized_features = array([[8.44432659e-03, 4.70305017e-07, 2.54895802e-04, ...,
 1.26982355e-05, 5.22538579e-04, 5.22538579e-04],
 [8.39877977e-03, 4.70256426e-07, 1.22132863e-03, ...,
 1.26969235e-05, 2.09441972e-02, 2.09441972e-02],
 [8.35414495e-03, 4.70258652e-07, 1.22047882e-03, ...,
 1.26969836e-05, 1.96273405e-02, 1.96273405e-02],
 ...,
 [4.44036266e-03, 4.76945506e-07, 9.90913953e-05, ...,
 1.28775287e-05, 5.02814322e-04, 5.02814322e-04],
 [4.39249471e-03, 4.76667901e-07, 7.82743947e-04, ...,
 1.28700333e-05, 1.74738778e-02, 1.74738778e-02],
 [4.34833193e-03, 4.76790782e-07, 7.12455944e-04, ...,
 1.28733511e-05, 6.36197369e-03, 6.36197369e-03]], shape=(94, 56))
learning_time = 0.007094299999153009
model.cluster_centers_ = array([[5.32414317e-03, 4.74945867e-07, 1.27453324e-04, ...,
 1.28235384e-05, 5.26628361e-03, 5.26628361e-03],
 [5.06003295e-03, 4.75567007e-07, 8.96371498e-04, ...,
 1.28403092e-05, 1.79477539e-02, 1.79477539e-02],
 [4.83803808e-03, 4.75950622e-07, 6.76062049e-04, ...,
 1.28506668e-05, 1.81235509e-02, 1.81235509e-02],
 ...,
 [8.09638256e-03, 4.70856793e-07, 1.27811936e-03, ...,
 1.27131334e-05, 1.97463590e-02, 1.97463590e-02],
 [6.34618833e-03, 4.73772925e-07, 6.43811415e-04, ...,
 1.27918690e-05, 7.78692045e-03, 7.78692045e-03],
 [5.19271064e-03, 4.75305322e-07, 8.46694149e-04, ...,
 1.28332437e-05, 2.24362166e-02, 2.24362166e-02]], shape=(57, 56))

```

Рисунок 1.1 — Виконання, частина 1


```

result = array([17, 50, 3, 24, 38, 56, 12, 46, 32, 27, 11, 25, 8, 52, 44, 30, 31,
51, 47, 22, 40, 5, 31, 31], dtype=int32)
prediction_time = 0.0006795999979658518
first_prediction_error = -0.006499325323391448
first_error_probability = -0.003899595194034869
first_right_probability = 1.0038995951940348
new_features = [3.5149749364445397, 0.9895495366028397, 4.67411035039745, 6.9254713421849425, 11.23
7420703160048, 18.27440596468842, 7.76759318659341, 16.60094054438537, 17.447541048621442, 5.321312
391899397, 15.124930171634867, 2.6355415105162674, 12.305136298586358, 16.351306179375772, 11.87118
7302623081, 15.066205622553007, 8.20433005528686, 14.491086465279832, 3.3844586093762405, 7.8219874
80452135, 18.25779406123037, 12.650557995957318, 4.371511219020834, 10.076203780519638, 8.799269123
81211, 2.749994954678458, 5.487323661151519, 8.39773087788021, 15.917703064611953, 12.2664055836807
86, 9.225861649444235, 1.4117339992556612, 16.903604336926136, 8.9163505840772, 3.3711417371398142,
18.242952747918164, 1.6628575423053444, 2.4680494597555027, 12.650618676014979, 1.0747082801912122
, 6.6528501363492945, 2.289989604350588, 11.832929485615175, 1.1255789451077418, 14.425919188848416
, 4.154959117288836, 15.829838837820795, 8.759332373242493, 3.032201920241801, 1.5156879682883033,
0.5781840495405618, 7.995239494849987, 10.905016820686999, 0.19890230538818543, 10.41755396213082,
4.2047090147054735, 4.588829866413553, 12.066013851030586, 10.636644129676815, 1.145782074331809, 1
1.725683558445715, 3.584128387520791, 5.886776626298962, 1.5351365392405096, 3.740989265662405, 15.
286970958941462, 11.648523004615955, 8.70855559360987, 2.66091031149852, 0.5205893021201451, 18.911
976817912922, 3.2081051605901485, 10.222740300941012, 7.4952344964544695, 18.143565218113135, 17.86
1298237653564, 9.121210964673196, 17.546133338095526, 12.134719650457068, 1.1456205647538762, 16.57
112809849916, 6.217548103983447, 4.504254056175925, 9.758028131994784, 16.794690991242348, 16.23326
061597711, 13.439187612337019, 11.709932724452234, 13.33891177194819, 11.72131154953775, 12.4880068
88548805, 10.282150630220478, 6.285638968213807, 2.6003450380271125]
new_center = 22
new_target = 0
result = array([13, 5, 33, 38, 37, 9, 41, 40, 16, 11, 8, 28, 22, 25, 35, 8, 46,
19, 48, 41, 9, 45, 6, 55, 7, 1, 50, 30, 15, 22, 23, 17, 2, 54,
48, 9, 56, 51, 45, 43, 3, 39, 35, 43, 19, 42, 15, 7, 34, 17, 26,
12, 20, 18, 49, 42, 33, 44, 36, 43, 10, 13, 29, 17, 47, 53, 10, 7,
28, 26, 24, 52, 0, 27, 9, 32, 23, 16, 44, 43, 40, 21, 6, 31, 2,
25, 14, 10, 14, 10, 4, 0, 21, 28], dtype=int32)
prediction_error = -0.09603694637157342
error_probability = -0.001021669642250781
right_probability = 1.0010216696422507
prediction_error_difference = 0.08953762104818197
error_probability_difference = 0.002877925551784088
right_probability_difference = 0.002877925551784122

```

Рисунок 1.2 — Виконання, частина 2

У другій вибірці помилка більше. В ній також менше екземплярів. Тому, можливо, чим більше вибірка — тим менше помилка.

Частина 2

Вибірка є даними про пасажирів човна Титанік, що потонув; має такі ознаки:

- PassengerId: ідентифікатор;
- Survived: 0 — не вижив, 1 — вижив;

- Pclass: клас, 1 або 2 або 3;
- Name: ім'я пасажира;
- Sex: male або female;
- Age: вік;
- SibSp: кількість братів/сестер або дружини на борту;
- Parch: кількість батьків або дітей;
- Ticket: номер квитка;
- Fare: вартість квитка;
- Cabin: каюта;
- Embarked: порт, на якому сіли: С — Чербург, Q — Квінстаун, S — Саутхемптон;

Пропущені дані є у стовпці віку. Їх було замінено на середнє значення віку. Відкидати рядки не став, бо чим більше вибірка, тим краще.

```
import os
import time
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder, normalize
from sklearn.svm import SVC

current_folder = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(current_folder, '..', 'train.csv')
df = pd.read_csv(file_path)

# Remove unnecessary columns
unnecessary='PassengerId,Pclass,SibSp,Parch,Ticket,Fare,Cabin,Embarked'
df=df.drop(unnecessary,axis=1)
print(df)

# Remove name
df=df.drop('Name',axis=1)
# Count missing values
```

```

print(df.isnull().sum())

# Convert gender to 0 and 1
le=LabelEncoder()
df['Sex']=le.fit_transform(df['Sex'])

# Replace missing ages with average
imputer=KNNImputer(n_neighbors=7)
df['Age']=imputer.fit_transform(df[['Age']]).ravel()
print(df)

# Split dataset
target_feature=df.Survived.to_numpy()
survived_scores=pd.DataFrame(target_feature,columns=['Survived'])
df=df.drop(['Survived'],axis=1)
X_train,X_test,y_train,y_test=train_test_split(df,target_feature)

# Modeling results
kmeans_model=KMeans()
kmeans_model.fit(X_train)
kmeans_results=kmeans_model.predict(X_test)
print(f'{kmeans_model.cluster_centers_ = }')
print(f'{kmeans_results = }')

kneighbors_model=KNeighborsClassifier(n_neighbors=7)
kneighbors_model.fit(X_train,y_train)
kneighbors_results=kneighbors_model.predict(X_test)
print(f'{kneighbors_results = }')

forest_model=RandomForestClassifier()
forest_model.fit(X_train,y_train)
forest_results=forest_model.predict(X_test)
print(f'{forest_results = }')

logistic_model=LogisticRegression()
logistic_model.fit(X_train,y_train)
logistic_results=logistic_model.predict(X_test)
print(f'{logistic_results = }')

svc_model=SVC()
svc_model.fit(X_train,y_train)
svc_results=svc_model.predict(X_test)
print(f'{svc_results = }')

# Comparing models
testing_indeces=list(X_test.index)
corrects=[int(survived_scores.iloc[i].Survived) for i in
testing_indeces]

# Plotting graphs
models=['KNeighbors','RandomForest','Logistic','SVC']
accuracies=[sum([int(abs(corrects[i]-model[i])) for i in
range(len(corrects))]) for model in
[kneighbors_results,forest_results,logistic_results,svc_results]]
print("Errors in each model:",{m:a for m in models for a in
accuracies})
plt.title('How unaccurate is each model')
plt.bar(models,accuracies,color='g')

```

```
plt.show()
```

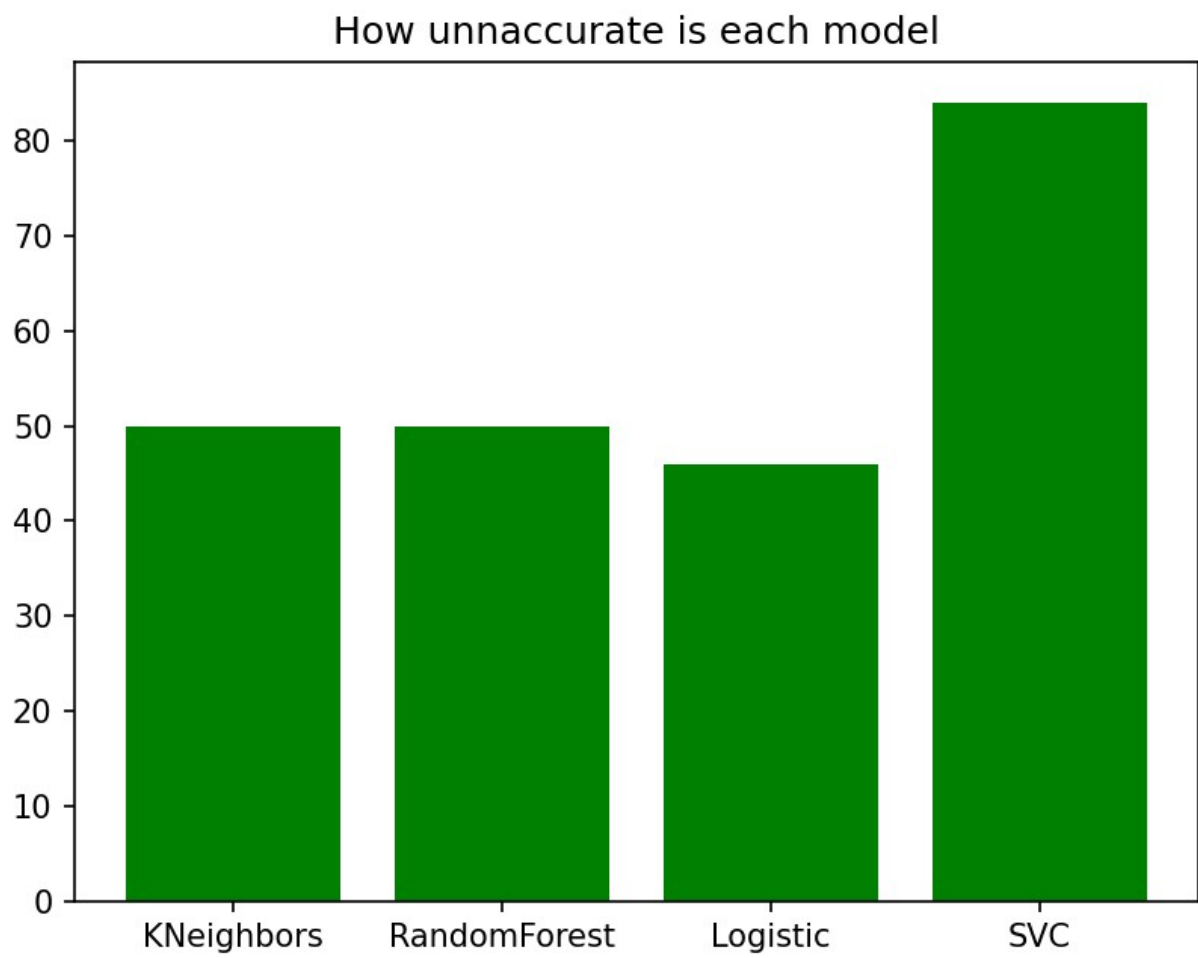


Рисунок 2.1 — Результати порівняння

	Survived	Name	Sex	Age
0	0	Braund, Mr. Owen Harris	male	22.0
1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
2	1	Heikkinen, Miss. Laina	female	26.0
3	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
4	0	Allen, Mr. William Henry	male	35.0
..	...	...	...	...
886	0	Montvila, Rev. Juozas	male	27.0
887	1	Graham, Miss. Margaret Edith	female	19.0
888	0	Johnston, Miss. Catherine Helen "Carrie"	female	NaN
889	1	Behr, Mr. Karl Howell	male	26.0
890	0	Dooley, Mr. Patrick	male	32.0

[891 rows x 4 columns]

Survived 0

Sex 0

Age 177

dtype: int64

	Survived	Sex	Age
0	0	1	22.000000
1	1	0	38.000000
2	1	0	26.000000
3	1	0	35.000000
4	0	1	35.000000
..	...	...	...
886	0	1	27.000000
887	1	0	19.000000
888	0	0	29.699118
889	1	1	26.000000
890	0	1	32.000000

[891 rows x 3 columns]

```
kmeans_model.cluster_centers_ = array([[ 0.67322835, 29.56173923],
    [ 0.71428571, 58.44285714],
    [ 0.44680851, 14.35106383],
    [ 0.63636364, 46.50757576],
    [ 0.65185185, 21.55185185],
    [ 0.58823529, 37.15882353],
    [ 0.46153846,  3.59205128],
    [ 1.          , 71.71428571]])
```

Рисунок 2.2 — Результати виконання, частина 1

```

kmeans_results = array([4, 6, 0, 0, 4, 5, 5, 3, 0, 4, 0, 3, 0, 4, 0, 0, 1, 0, 0, 4, 5, 5,
0, 5, 6, 6, 4, 0, 6, 6, 0, 0, 0, 5, 3, 3, 4, 0, 4, 0, 4, 0, 4, 6,
0, 0, 3, 0, 0, 0, 3, 3, 0, 5, 4, 2, 1, 5, 2, 6, 4, 0, 0, 4, 0, 5,
0, 0, 4, 0, 6, 4, 0, 4, 0, 0, 0, 4, 4, 0, 5, 0, 2, 4, 0, 4, 0, 5,
4, 5, 0, 0, 4, 2, 1, 3, 4, 5, 1, 0, 0, 0, 3, 0, 0, 3, 0, 0, 5, 0,
1, 2, 0, 5, 6, 0, 4, 0, 0, 0, 4, 3, 0, 2, 6, 5, 1, 0, 0, 0, 0, 3,
0, 0, 6, 0, 0, 6, 0, 4, 2, 4, 4, 1, 4, 4, 4, 5, 4, 0, 3, 0, 4, 0,
4, 5, 4, 4, 3, 6, 4, 4, 0, 3, 0, 3, 0, 0, 4, 0, 0, 5, 4, 4, 4,
3, 2, 1, 3, 5, 5, 2, 0, 5, 3, 0, 3, 0, 4, 4, 0, 4, 2, 5, 0, 4, 4,
0, 4, 0, 4, 3, 0, 4, 0, 0, 3, 4, 7, 3, 5, 2, 4, 6, 0, 6, 4, 0, 3,
3, 0, 2], dtype=int32)
kneighbors_results = array([1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0,
0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0,
0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1,
0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1,
0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1,
0, 0, 0])
forest_results = array([1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0,
0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0,
0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1,
0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1,
0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1,
0, 0, 0])
logistic_results = array([1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0,
0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0,
0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1,
0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1,
0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0,
0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1,
0, 0, 0])
svc_results = array([0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0])
Errors in each model: {'KNeighbors': 84, 'RandomForest': 84, 'Logistic': 84, 'SVC': 84}

```

Рисунок 2.3 — Результати виконання, частина 2

Логістична регресія завжди має кращі результати. Після неї випадковий ліс і k-сусідів. SVC завжди остання.

Покращити результати можна збільшивши тестувальну вибірку.

ПИТАННЯ

Задача розпізнавання образів.

Віднесення даних до якогось класу в залежності від ознак.

Основні поняття теорії розпізнавання образів.

Навчання — процес побудови моделі.

Ознака — характеристика набору даних.

Розпізнавання — отримання результатів моделі.

Розбиття вихідної вибірки на навчаючу та тестову.

Потрібне, щоб модель могла працювати з ще не баченими даними.
Перемішує дані і розділяє їх на два масиви різних розмірів.

Навчання з учителем.

Це коли моделі задаються правильні значення вихідної ознаки.

Методи метричної класифікації.

Модель створює кластери, шукає координати еталонів (середніх екземплярів) кластеру, категоризує вхідні екземпляри в залежності від вихідної ознаки до найближчого центру.

Призначення пакету «Sklearn».

Машинне навчання та розпізнавання образів.

Що таке генеральна сукупність, вибірка, екземпляр, ознака?

Генеральна сукупність — всі об'єкти якогось виду.

Вибірка — частина генеральної сукупності, яка представляє її точно.

Екземпляр — один з об'єктів вибірки.

Ознака — якась властивість екземпляру: колір.

Вимоги до навчаючих вибірок даних.

1. Репрезентативність;
2. Розмір хоча б 100 екземплярів;
3. Відсутність пропущених значень;
4. Текстові дані або прибрані, або переведені у мітки.

Що таке репрезентативна вибірка даних?

Така, що достовірно представляє дані.

Чи повинна навчальна вибірка бути репрезентативною?

Так.

Чи повинна тестова вибірка бути репрезентативною?

Краще щоб була.

Чи впливає обсяг навчаючої вибірки на швидкість навчання?

Так.

Чи впливає репрезентативність навчаючої вибірки на точність класифікації екземплярів тестової вибірки?

Так.

Чи впливає репрезентативність тестової вибірки на точність класифікації екземплярів тестової вибірки?

Так, впливає, але не так, як репрезентативність навчальної.

Чи впливає репрезентативність тестової вибірки на точність навчання перцептрона за навчаючої вибіркою?

Ні, репрезентативність тестової вибірки впливає лише на результати розпізнавання.

Чи залежить якість навчання від якості та обсягу навчаючої вибірки?

Так, залежить. Якщо обсяг недостатній, помилка може бути більшою. Якщо якісь дані відсутні, моделі не зможуть опрацювати їх.